Documentation zur Prüfungsleistung Network Security

Cyber Security | DHBW Mannheim | 2958274

9. November 2026

1 Einleitung

Die Prüfungsleistung in Network Security gibt vor zwei Programme zu schreiben, die eine Datenextrahierung über eine der folgendenden vier Protokollen: DNS, ICMP, ARP und SNMP zu erstellen. Dabei muss die Exfiltration aus einer Datei erfolgen und mittels des Protokolls auf einen Remote-Server in einem IPv4 Subnetz übertragen werden. Die Daten müssen dabei mithilfe einer Encodierung übertragen werden. Der Server empfängt die Daten, baut sie zusammen, decodiert sie und schreibt sie anschließend in eine Datei. Beide seiten müssen dabei den Traffic aufzeichnen und in eine pcap-datei schreiben.

2 Motivation

Angreifer nutzen ICMP trotz seiner grundsätzlich einfachen Filterbarkeit vor allem deshalb, weil das Protokoll in der Praxis häufig unzureichend kontrolliert wird. Während eingehender Verkehr vielerorts sorgfältig nach dem Prinzip Default-Deny abgesichert wird, bleibt die ausgehende Filterung (Egress) oft offen konfiguriert, sodass ICMP das Netz ungehindert verlassen kann [1,2]. Ein wesentlicher Faktor ist dabei, dass ICMP keinen Port verwendet, sondern als eigener Protokolltyp der Netzwerkschicht arbeitet, wodurch rein portbasierte Firewall-Regeln es implizit unberücksichtigt lassen [3]. Als vermeintlich harmloser Diagnoseverkehr wird ICMP zudem seltener protokolliert oder inhaltlich inspiziert als die klassischen Exfiltrations-Kandidaten HTTP und DNS, die heute regelmäßig durch Proxies und Deep Packet Inspection geprüft werden [1,4]. Ein in der Nutzlast von Echo-Request- und Echo-Reply-Paketen versteckter Tunnel fällt daher kaum auf, zumal auf Header-Ebene kein Unterschied zu legitimem Ping besteht; die Daten werden schlicht in den Payload-Bereich der ICMP-Pakete eingebettet [5,6]. Da ein geringes Grundrauschen an ICMP in nahezu jedem Netz normal ist, blendet sich solcher Verkehr unauffällig in den regulären Betrieb ein und lässt sich erst über Anomalien wie ungewöhnlich große oder stark schwankende Paketgrößen erkennen [6,7]. Für Command-and-Control-Kommunikation genügt die geringe Bandbreite eines ICMP-Kanals vollständig; für die Exfiltration ist er mit etwa einem Byte Nutzlast pro Paket zwar ineffizient, eignet sich aber als unauffälliger und langlebiger Fallback-Kanal, falls primäre Wege wie HTTPS blockiert werden [1,8]. Dass es sich dabei nicht um ein theoretisches Restrisiko handelt, zeigt der Einsatz in realen Kampagnen: Die staatlich zugeschriebene Gruppe Turla setzte protokollbasierte Covert Channels – darunter ICMP-Tunneling – als verdeckten Kanal ein [9], und die Grundidee reicht mit Werkzeugen wie Loki bis in die 1990er-Jahre zurück [4,10]. Entscheidend für die Attraktivität von ICMP ist somit nicht eine technische Überlegenheit des Protokolls, sondern der Umstand, dass es in vielen Umgebungen schlicht vergessen und nicht überwacht wird – ein Vorteil, der sich durch konsequente Egress-Filterung und gezieltes Monitoring neutralisieren lässt [2,7].

3 Technische Beschreibung von ICMP

3.1 Aufbau eines ICMP-Pakets

ICMP ist ein eigenständiges Protokoll der Vermittlungsschicht (Layer 3) und wird direkt in IP-Paketen gekapselt; im IPv4-Header kennzeichnet die Protokollnummer 1 die ICMP-Nutzlast [12]. Anders als TCP oder UDP besitzt ICMP keine Transportschicht und damit keine Portnummern. Ein ICMP-Paket besteht aus dem IP-Header, gefolgt vom ICMP-Header mit den Feldern Type, Code und Checksum sowie – im Fall von Echo Request (Type 8) und Echo Reply (Type 0) – einem Identifier- und einem Sequence-Number-Feld. Daran schließt sich das Datenfeld (Payload) an, das beim regulären Ping lediglich mit Füllbytes belegt ist [12].

3.2 Einbettung der Nutzdaten

Genau dieses Datenfeld dient als Trägerbereich für die Exfiltration: Statt der üblichen Füllbytes bettet die Malware die zu übertragenden Daten in den Payload von Echo-Request-Paketen ein [5, 6]. Da ICMP keine eigene Sitzungslogik mitbringt, implementiert der Angreifer eine etwaige Reihenfolge- und Wiederzusammensetzungslogik selbst, etwa über Sequenznummern oder eine eigene Chunk-Kennung im Payload. Kleine Datenmengen lassen sich zusätzlich in den Identifier- und Sequence-Number-Feldern verstecken.

3.3 Weg durch den Netzwerkstack

Der kompromittierte Host liest die Daten, zerlegt sie in Blöcke passender Größe und erzeugt über einen Raw Socket rohe ICMP-Echo-Request-Pakete, die an die IP-Adresse des angreifergesteuerten Servers gerichtet sind. Router und Firewalls behandeln diese Pakete wie regulären Ping-Verkehr und leiten sie weiter. Der Server muss lediglich auf eingehende ICMP-Pakete lauschen (ebenfalls per Raw Socket bzw. `scapy` in python), den Payload extrahieren und die Blöcke wieder zusammensetzen. Antwortet der Server mit Echo-Reply-Paketen, deren Payload Steuerbefehle enthält, entsteht ein bidirektionaler, verdeckter Kanal (Command-and-Control) [3].

3.4 Größenbegrenzung der Nutzlast

Die übertragbare Datenmenge pro Paket ist durch die Maximum Transmission Unit (MTU) begrenzt – bei Ethernet typischerweise 1500 Byte. Abzüglich des IP- und ICMP-Headers verbleiben für ein Standard-Echo-Paket rund 1472 Byte Nutzdaten. Überschreitet die Nutzlast die MTU, wird das IP-Paket *fragmentiert* (bzw. bei gesetztem *Don't-Fragment*-Bit verworfen); es wird dadurch jedoch nicht als ein anderes Protokoll interpretiert. In der Praxis wird daher bevorzugt in viele kleine ICMP-Pakete aufgeteilt (Chunking), um Fragmentierung zu vermeiden und unauffällig zu bleiben. Ungewöhnlich große oder stark schwankende ICMP-Payloads gelten umgekehrt als zuverlässiges Erkennungsmerkmal, da reguläres Ping kleine, feste Paketgrößen verwendet [6].

4 Technische Beschreibung von Base64

Für die Aufbereitung der Nutzlast wird mit Base64 ein in RFC 4648 standardisiertes Transfer-Encoding eingesetzt [11]. Es bildet beliebige Binärdaten auf 64 druckbare Zeichen ab (`A-Z`, `a-z`, `0-9`, `+`, `/`), wobei jedes Zeichen wegen $2^6 = 64$ genau sechs Bit repräsentiert. Zur Codierung wird der Bytestrom in Blöcke zu je 24 Bit zerlegt und in vier 6-Bit-Gruppen unterteilt; jede Gruppe liefert einen Index von 0 bis 63, der über die Alphabettabelle auf ein Zeichen abgebildet wird, sodass aus drei Eingabebytes vier Ausgabezeichen entstehen. Ist die Eingabelänge kein Vielfaches von drei, wird der letzte Block mit Null-Bits aufgefüllt und mit dem Füllzeichen `=` gepolstert, das zugleich die Zahl der tatsächlichen Datenbytes kodiert.

Die Decodierung kehrt den Vorgang um: Jedes Zeichen wird auf seinen 6-Bit-Wert zurückabgebildet, die Gruppen werden zu einem fortlaufenden Bitstrom verkettet und erneut in 8-Bit-Bytes gruppiert, wobei die durch das Padding hinzugefügten Bits verworfen werden und exakt die ursprüngliche Bytefolge entsteht.

5 technische Beschreibung der möglichen Integritätsprüfung und Fehlerkorrektur

Auf Protokollebene bietet ICMP für die Zuverlässigkeit der übertragenen Daten kaum Mechanismen: Das 16-Bit-Checksum-Feld im ICMP-Header erlaubt lediglich eine Prüfung auf zufällige Bitfehler über eine Einerkomplement-Summe, bietet aber weder kryptographische Integrität noch eine Korrektur; ein als fehlerhaft erkanntes Paket wird je nach Netzwerkstack verworfen, nicht repariert [12]. Da ICMP zudem verbindungslos und vergleichbar mit UDP, unzuverlässig ist, können Pakete verloren gehen, dupliziert oder in veränderter Reihenfolge ankommen, ohne dass das Protokoll dies ausgleicht. Ein Exfiltrations-Tunnel muss Integritätssicherung und Fehlerkorrektur daher selbst in der Nutzlast implementieren. Zur Integritätsprüfung kommen je nach Schutzziel eine einfache Prüfsumme bzw. ein CRC pro Datenblock (erkennt Übertragungsfehler), ein HMAC oder ein AEAD-Verfahren wie AES-GCM (erkennt zusätzlich gezielte Manipulation und verschlüsselt zugleich) sowie ein abschließender Gesamt-Hash über die rekonstruierte Datei zur Ende-zu-Ende-Verifikation in Frage. Für die Fehlerkorrektur stehen zwei grundsätzliche Ansätze zur Verfügung. Beim rückkanalbasierten Automatic Repeat reQuest (ARQ) versieht der Sender jeden Block mit einer Sequenznummer, der Empfänger bestätigt korrekt empfangene Blöcke per ACK oder fordert fehlende per NACK an, und der Sender überträgt unbestätigte Blöcke nach einem Timeout erneut; die Rückmeldungen lassen sich dabei über die Echo-Reply-Pakete des bidirektionalen ICMP-Mechanismus transportieren. Je nach Variante, Stop-and-Wait, Go-Back-N oder Selective Repeat, unterscheiden sich Durchsatz und Pufferbedarf, wobei Selective Repeat den höchsten Durchsatz erzielt, da nur tatsächlich fehlende Blöcke wiederholt werden [13]. Die Forward Error Correction (FEC) verzichtet dagegen auf einen Rückkanal: Der Sender fügt den k Datenblöcken vorab Redundanz hinzu und überträgt $n > k$ kodierte Blöcke, sodass der Empfänger aus beliebigen k der n Blöcke die Originaldaten rekonstruieren kann, solange nicht mehr als $n-k$ Blöcke verloren gehen, etwa mittels Reed-Solomon- oder rateless Fountain-Codes. RFC 6363 definiert hierfür ein allgemeines Framework für den Schutz beliebiger Paketflüsse über unzuverlässige Transportwege [14]. Beide Verfahren erkaufen die gewonnene Zuverlässigkeit jedoch mit einem für die Tarnung nachteiligen Preis: ARQ erzeugt zusätzlichen Echo-Reply-Verkehr, FEC einen permanenten Bandbreiten-Overhead durch Redundanzpakete und gerade eine erhöhte Paketanzahl sowie ungewöhnlich große oder stark schwankende ICMP-Payloads zählen zu den zuverlässigsten Erkennungsmerkmalen für Intrusion-Detection-Systeme [6]. Robustheit und Unauffälligkeit stehen somit in einem direkten Zielkonflikt.

6 Fazit

Aus meiner Sicht ist ICMP-Tunneling eine in der Praxis durchaus erfolgreiche, zugleich aber stark kontextabhängige Angriffsmethode. Die Technik ist einfach zu implementieren, fertige Werkzeuge wie ptunnel oder Loki stehen bereit [10] und lässt sich mit den beschriebenen Mechanismen (eigene Sequenzierung, ARQ oder FEC bzw. vollständiges TCP-over-ICMP) ohne großen Aufwand robust gestalten. Ihr Erfolg hängt jedoch entscheidend von der Zielsetzung und der Konfiguration des Zielnetzes ab: Als verdeckter Command-and-Control- oder Fallback-Kanal mit geringem Bandbreitenbedarf ist ICMP zuverlässig und unauffällig [3], während er sich für die Exfiltration großer Datenmengen aufgrund der geringen Nutzlast pro Paket nur langsam und mit steigender Entdeckungswahrscheinlichkeit eignet, Robustheit und Tarnung stehen hier in direktem Konflikt.

Voraussetzung ist zudem, dass ein interner Host bereits kompromittiert ist, es handelt sich also um eine Post-Exploitation-Technik und keinen Erstzugang. Für das Erzeugen roher ICMP-Pakete sind erhöhte Rechte erforderlich (unter Linux die Capability `CAP_NET_RAW`, unter Windows in der Regel Administrator, in der Praxis meist Root), wobei je nach Konfiguration auch unprivilegierte ICMP-Echo-Sockets missbraucht werden können. Entscheidend für den Erfolg ist letztlich weniger die technische Raffinesse als die Nachlässigkeit der Verteidigung: In gut abgesicherten Netzen scheitert der Angriff, in schlecht überwachten gelingt er zuverlässig.

Erkennen lässt sich der Angriff vor allem über die Nutzlast: Intrusion-Detection- bzw. Network-Detection-Systeme schlagen bei ungewöhnlich großen oder starkschwankenden ICMP-Payloads an, die sich deutlich vom festen, kleinen Formatregulärer Ping-Pakete unterscheiden [6,7]. Weitere Indikatorensind eine hohe Paketfrequenz oder ein kontinuierlicher Strom zu einem einzelnen externen Host, eine auffällige Asymmetrie zwischen Requests und Replies sowie eine hohe Entropie im Payload als Hinweis auf Verschlüsselung. Reine Flow-Daten genügen dafür nicht, notwendig ist ein Einblick in Payload-Größe und -Inhalt (Deep Packet Inspection) auf Basis einer zuvor etablierten Baseline des Normalverkehrs. Verhindern lässt sich der Angriff primär durch eine konsequente Egress-Filterung nach dem Prinzip Default-Deny: ausgehender ICMP-Echo-Verkehr wird blockiert oder eng begrenzt (Rate-Limiting, Payload-Größe), nur benötigte ICMP-Typen wie Typ 3 für die Path-MTU-Discovery bleiben erlaubt, ergänzt um Netzsegmentierung und einen erzwungenen Proxy-Ausgang. Auf Host-Ebene wirken das Prinzip der minimalen Rechte (Entzug von `CAP_NET_RAW`, Einschränkung der Raw-Socket-Erstellung) sowie EDR-Lösungen gegen die zugrunde liegende Malware. Benötigt werden dafür eine Egress-fähige Firewall, ein IDS/NDR mit Payload-Inspektion sowie eine Logging- und Monitoring-Infrastruktur mit entsprechender Auswertung; eine zustandsbehaftete Firewall allein reicht nicht aus, da sie den von innen initiierten Tunnel nicht als bösartig erkennt.

Literatur

[1] Praetorian, "ICMP C2: Standard Non-Application Layer Protocol (ATT&CK T1095)," Praetorian Blog. Zugriff: 9. Nov. 2026. [Online]. Verfügbar: `https://www.praetorian.com/blog/icmp-c2-standard-non-application-layer-protocol/`

[2] M. Smeets und M. Koot, "Research Report: Covert Channels," MSc System and Network Engineering, University of Amsterdam, Feb. 2006. [Online]. Verfügbar: `https://www.os3.nl/_media/2005-2006/rp1/ms_mk_report.pdf`

[3] The MITRE Corporation, "Non-Application Layer Protocol, Technique T1095," MITRE ATT&CK. Zugriff: 9. Nov. 2026. [Online]. Verfügbar: `https://attack.mitre.org/techniques/T1095/`

[4] G. Merezas und L. Wüstrich, "Covert Communication over ICMP," in *Seminar IITM WS 23*, Lehrstuhl für Netzarchitekturen und Netzdienste, Technische Universität München, S. 77–81, 2024, doi: 10.2313/NET-2024-04-1_14. [Online]. Verfügbar: `https://www.net.in.tum.de/fileadmin/TUM/NET/NET-2024-04-1/NET-2024-04-1_14.pdf`

[5] The MITRE Corporation, "Protocol Tunneling, Technique T1572," MITRE ATT&CK. Zugriff: 9. Nov. 2026. [Online]. Verfügbar: `https://attack.mitre.org/techniques/T1572/`

[6] Elastic, "Potential ICMP Tunneling Activity to the Internet," Elastic Security Detection Rules, v8.19. Zugriff: 9. Nov. 2026. [Online]. Verfügbar: `https://www.elastic.co/guide/en/security/8.19/potential-icmp-tunneling-activity-to-the-internet.html`

[7] Vectra AI, "ICMP Tunnel: Server," Vectra AI Detections. Zugriff: 9. Nov. 2026. [Online]. Verfügbar: `https://www.vectra.ai/detections/icmp-tunnel-server`

[8] The MITRE Corporation, "Exfiltration Over Alternative Protocol, Technique T1048," MITRE ATT&CK. Zugriff: 9. Nov. 2026. [Online]. Verfügbar: https://attack.mitre.org/techniques/T1048/

[9] The MITRE Corporation, "Turla, Group G0010," MITRE ATT&CK. Zugriff: 9. Nov. 2026. [Online]. Verfügbar: https://attack.mitre.org/groups/G0010/

[10] daemon9, "Project Loki," *Phrack Magazine*, Bd. 7, Ausg. 49, 1996. [Online]. Verfügbar: http://phrack.org/issues/49/

[11] S. Josefsson, "The Base16, Base32, and Base64 Data Encodings," IETF, RFC 4648, Oct. 2006. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4648

[12] J. Postel, "Internet Control Message Protocol," RFC 792, Internet Engineering Task Force, Sep. 1981. [Online]. Verfügbar: https://www.rfc-editor.org/rfc/rfc792

[13] S. Lin, D. J. Costello und M. J. Miller, "Automatic-repeat-request error-control schemes," *IEEE Communications Magazine*, Bd. 22, Nr. 12, S. 5–17, Dez. 1984, doi: 10.1109/MCOM.1984.1091865.

[14] M. Watson, A. Begen und V. Roca, "Forward Error Correction (FEC) Framework," RFC 6363, Internet Engineering Task Force, Okt. 2011, doi: 10.17487/RFC6363. [Online]. Verfügbar: https://www.rfc-editor.org/rfc/rfc6363